
COMMON FOUNDRY

Open Compute, Verifiable Work, Direct Settlement

Technical Whitepaper | Version 0.3

September 7, 2026

Matrix-oriented proof of work, transparent verification, and a direct settlement foundation for GPU services.

384

sequential layers

6.093 s

RTX 5090 online proof

RCNet-1

live release candidate

A technical foundation for an open operator ecosystem. Measured software, consumer GPUs, and a defined path to service-market deployment.

Inside this edition

1. Executive thesis
2. Product architecture and adoption strategy
3. ForgeMatrix: the active computation
4. Transparent proof and independent verification
5. Demonstrated performance and RC operations
6. Ledger, network, and operator software
7. CMFD monetary design
8. Direct inference settlement
9. Engineering assurance and execution roadmap
10. Conclusion

Appendix A. Technical parameter reference

Appendix B. Evidence and source notes

Reading guide

Sections 1-2 explain the commercial thesis and adoption sequence. Sections 3-6 describe the active technology and operating evidence. Sections 7-9 cover monetary design, service settlement, and execution. The appendices collect protocol parameters and source notes.

1. Executive thesis

Common Foundry is building infrastructure for a GPU economy in which operators can contribute verifiable work and, as the service layer develops, sell inference directly to customers. The architecture combines three complementary mechanisms: matrix-oriented mining to secure ledger ordering, a CMFD settlement asset with explicit monetary rules, and cumulative payment channels for metered GPU services.

The strategic advantage is the relationship between the hardware ecosystem and the protocol. ForgeMatrix concentrates mining on dense matrix computation, a workload closely associated with modern machine-learning hardware and software. Operators develop experience with GPU deployment, memory management, kernels, power efficiency, and availability. These capabilities also matter when delivering commercial inference. The proposed service layer gives that operator base a second application for its infrastructure.

Consensus and inference have separate responsibilities. The public ForgeMatrix relation supplies deterministic block validity. Customers select models and providers through a service market, with payment governed by signed authorizations. This division keeps block production independent of customer demand and allows the service layer to evolve models, runtimes, and commercial terms on its own cadence.

What is differentiated

Technology choice	Mechanism	Strategic significance
Matrix-oriented proof of work	A fixed 384-layer matrix computation bound to each block candidate	Develops an operator ecosystem around GPU compute rather than a narrow hash-only workload
Transparent verification	GPU BaseFold proof with CPU candidate verification and independent node admission	Makes computation auditable while separating miner performance from verification authority
Consumer hardware participation	Qualified proof path on an RTX 5070 Ti 16 GB; separate pool-search clients	Creates multiple participation paths with different hardware and setup requirements
Direct service settlement	Prepaid channels with cumulative customer-signed authorizations	Supports granular service delivery without an on-chain transaction for every output chunk
Explicit network economics	Declining bootstrap issuance, visible allocations, fee burning, miner-only tail	Gives operators and ecosystem participants a model they can calculate from protocol rules

The near-term execution focus is network operation, mining efficiency, software distribution, and operator adoption. The longer-term commercial opportunity is a service ecosystem built on this installed technical foundation. The inference settlement primitives are implemented; discovery, job execution, and customer-facing marketplace workflows form the next product layer.

Evidence of execution

RC5 ships wallet and node software with integrated solo mining and a network-enforced transaction burn. The subsequent pool-miner update improves measured search throughput while preserving consensus outputs. The community pool began operating on September 6, 2026. A September 7 observation recorded 66 pool blocks and 7,836 accepted shares. This is an early operating milestone with a public dashboard, release artifacts, and hardware-specific measurements. [S1-S4]

RCNet-1 is the release-candidate environment. Mainnet is planned as a fresh network with an announced launch sequence; RC balances do not transfer. Common Foundry has no token sale.

2. Product architecture and adoption strategy

2.1 An operator-first route to market

The initial users are GPU owners, miners, pool operators, developers, and infrastructure partners. A usable miner, visible pool statistics, a functioning wallet, and repeatable setup are the first adoption surfaces. They give participants a concrete reason to install the software, evaluate its performance, and contribute improvements.

The proposed inference market builds on that participation rather than requiring a complete two-sided service market at network launch. Its development sequence is explicit:

1. **Establish the network.** Demonstrate verifiable mining, settlement, software upgrades, and sustained operator participation.
2. **Improve operator economics.** Optimize search and proving, broaden hardware qualification, and reduce setup and operating friction.
3. **Integrate service delivery.** Connect the implemented payment-channel rules to quotes, job transport, inference execution, and wallet workflows.
4. **Develop customer demand.** Start with a defined model/runtime combination and measured service quality, then expand through provider and application integrations.

This sequencing concentrates early engineering on a working product and makes each expansion measurable. For an inference pilot, useful indicators include time to first output, cost per completed job, delivery success, provider utilization, and repeat customer use. For the current network, accepted work, block propagation, recovery behavior, and energy-normalized performance are the primary operating indicators.

2.2 Four technical layers

Layer	Responsibility	Interface
Ledger and consensus	UTXO ownership, signatures, difficulty, issuance, fee burning, channel spends	Canonical transactions and blocks
ForgeMatrix	Block-bound computation, work digest, transparent proof	Mining templates and independently verified candidates
Service settlement	Channel terms, metering, signed authorizations, settlement and refund	Customer/provider states and consensus channel locks
Operator software	Nodes, wallets, solo miners, pool clients, pool accounting	Windows/Linux packages, local RPC, authenticated pool sessions

The ledger is the shared settlement foundation. GPU performance remains an implementation choice, while canonical verification determines acceptance. Service applications consume the channel rules through a separate product interface.

2.3 Participation paths

Pool mining gives operators a focused search client. The current client acquires approximately 6.4 GB of authenticated model data; the pool handles the full winner-proof and block-submission path. This reduces client setup compared with solo proving and provides frequent accepted-share feedback.

Solo mining combines search with the full proving stack and node interaction. RC5 setup acquires roughly 61 GB of model and preprocessed proving inputs, plus working storage. It supports operators who want to run the complete candidate-production path.

Node and wallet operation provides independent validation, ownership, and settlement. Wallet-only setup acquires roughly 6.4 GB of model data. The GUI exposes balances, send/receive, encrypted key operations, backup and restore, and mining controls. [S1-S2]

3. ForgeMatrix: the active computation

3.1 Geometry and arithmetic

ProductionV4 uses the ForgeMatrix geometry of 128 batch rows, 4,096 matrix dimensions, and 384 sequential layers. The layers are organized into three banks of 128. The public weight bank contains 6,442,450,944 raw bytes, exactly 6 GiB, plus a 524,288-byte base table and artifact framing.

The active arithmetic is over the KoalaBear prime field:

$$F = GF(p), \quad p = 0x7f000001 = 2,130,706,433$$

$$EF = F[X] / (X^4 - 3)$$

$$B = 128; \quad D = 4,096; \quad L = 384$$

Model bytes are drawn from 0..250 and mapped through $x - 125$ into the field. The initial activation is the cube of the base input plus a challenge-derived coordinate mask. Each layer applies a matrix product, adds its challenge-derived mask, and cubes the result elementwise:

$$A_0 = (\text{Base} + \text{Mask_initial}(\text{challenge}))^3$$

$$Z_1 = A_1 * W_1 + \text{Mask}_1(\text{challenge})$$

$$A_{(l+1)} = Z_1^3$$

All operations in these equations use canonical KoalaBear field semantics. The exact layout, mask derivation, transcript order, and encoding are fixed by the ProductionV4 specification and proof-system identity. Earlier integer-range and quotient/remainder constructions belong to the design lineage; the active V4 relation is the field-native matrix-and-cubic recurrence above. [S5-S6]

3.2 Why this workload matters

The geometry contains:

$$L * B * D * D = 824,633,720,832 \text{ matrix multiply-accumulate terms}$$

$$(L + 1) * B * D = 201,850,880 \text{ cubic activation outputs}$$

These are logical workload counts. Hardware kernels realize the field computation through their chosen exact implementation. The matrix shape offers substantial reuse of weights and activations, while sequential cubic transitions carry each layer into the next. The committed model bank supplies a shared, authenticated data resource.

This design gives optimization work a consistent target: the same computation, the same output, and the same verification rules across implementations. Improvements in batching, field arithmetic, memory movement, and hashing can increase throughput without redefining valid work.

Hardware competition remains open. Any implementation producing the required relation and proof can participate. Consumer GPUs are the current deployment focus, and physical-device qualification establishes the supported configurations.

3.3 Candidate binding

The challenge binds the full network identifier, previous block, transaction root, height, timestamp, target, algorithm version, proof version, proof-system digest, model-manifest digest, and nonce. The transaction root includes the coinbase commitment, binding reward outputs to the candidate.

The final activation contains 524,288 canonical field values in row-major order. Its domain-separated BLAKE3 digest includes the challenge and field count. A separate work digest binds the algorithm, proof system, model, challenge, and

final-activation digest. Acceptance requires that work digest to meet the independently derived chain target.

Proof randomness is excluded from the work digest. A candidate's work identity follows the deterministic computation, and the proof establishes its validity. This preserves a clear division between nonce search and winner verification. [S5]

4. Transparent proof and independent verification

4.1 Turning a large computation into a verifiable statement

ProductionV4 uses a GPU-native BaseFold path to prove the matrix, shift, and cubic relations. The proof exploits the algebraic structure of the workload, so validators verify a structured argument rather than repeat every matrix term.

Each bank carries a fixed weight commitment and a dynamic execution commitment. The verifier checks two relation repetitions per bank. The matrix relation binds the preactivation to the weighted input and public mask. The shift relation connects successive layers and bank boundaries. The cubic relation binds the next activation to the cube of the preactivation.

Opening claims connect those relations to committed data. Each bank receives 12 claims and pads to 16 by repeating the last claim in the specified order. The final bank is linked to the public final activation. This gives one end-to-end statement from model and challenge to terminal output. [S6]

4.2 Proof parameters and transcript

Parameter	ProductionV4 value
Base field	KoalaBear, 0x7f000001
Extension	Degree 4, $X^4 - 3$
Challenger	KoalaBearDegree4Duplex; Poseidon2 width 16, digest 8
Fixed / dynamic columns	256 / 16
Row variables	23
BaseFold log blowup	1
BaseFold queries	270
FRI rounds	23
Relation repetitions per bank	2
Batch grinding / proof-of-work grinding	5 / 16 bits
Opening claims per bank after padding	16

The Fiat-Shamir transcript begins with a digest of the complete public statement. Commitment order, relation order, claim routing, challenge derivation, and field encoding are consensus-critical. The proof-system digest commits these choices and the pinned BaseFold dependency revision. Transparent parameters and public artifacts support independent reproduction without secret setup material.

The matrix sumcheck uses 19 variables and degree 3; the shift sumcheck uses 7 variables and degree 2; the cubic sumcheck uses 26 variables and degree 4. Opening reduction uses a 23-variable, degree-2 sumcheck before BaseFold verifies the corresponding polynomial openings. [S6]

4.3 Candidate-production pipeline

1. The miner obtains a template and derives the block-bound challenge.
2. GPU search evaluates candidate nonces and their work digests.
3. A winning nonce enters the full replay and proof-construction path.
4. Mandatory CPU self-verification checks the completed candidate before submission.
5. Receiving nodes authenticate their artifacts, parse canonical objects, independently verify the proof, and admit the block into ledger state.

The CPU check is a design asset: it makes accelerator optimization replaceable while retaining one canonical source of acceptance rules. The same principle applies to pool operation, where the client search path and server proof path have different resource requirements.

4.4 Proof size and operating envelope

The measured transparent proof is exactly 12,025,320 bytes. It contains a 16-byte header, 2,097,152 bytes of public final-activation values, and three 3,309,384-byte bank sections. RCNet-1 provides a 13 MiB complete proof-frame limit and a 16 MiB block-frame limit.

At a hypothetical continuously full 16 MiB block every 60 seconds, inbound block payload is approximately 2.24 Mbit/s and daily data growth is approximately 22.5 GiB before database overhead. Actual blocks use their encoded size. Proof compression and propagation optimization are therefore concrete opportunities to improve operator efficiency as participation grows.

5. Demonstrated performance and RC operations

5.1 Full-proof qualification baselines

The following measurements are the previously recorded ProductionV4 qualification baselines. They describe distinct paths and retain their original measurement boundaries. [S7]

Hardware	Measured path	Result
RTX 5090	Winning replay	0.471 seconds
RTX 5090	Online proof construction	6.093 seconds
RTX 5090	Mandatory CPU self-verification	0.311 seconds
RTX 5090	Peak proving allocation	8.080 GiB
RTX 5070 Ti 16 GB	Complete three-process proof path	74.294 seconds, CPU-verified
RTX 5070 Ti 16 GB	Peak replay / proving allocation	13.730 GiB / 8.004 GiB

The RTX 5090 baseline demonstrates fast online winner proving. The physical RTX 5070 Ti result establishes correctness and memory fit in a 16 GB consumer tier. Setup, artifact preparation, and complete template-to-acceptance latency remain separate metrics from online proof time.

5.2 Pool-search improvement

The September 6 pool-miner update improved batching and CPU activation hashing while preserving intermediate replay outputs, final activations, and consensus digests. The published comparisons used the same hardware with unchanged clocks and power caps. [S2]

GPU	Prior console rate	Updated console rate	Measured change
RTX 4070 Ti SUPER	8.33 FW/s	20.285 FW/s	2.44x
RTX 5070 Ti	11.29 FW/s	27.59 FW/s	2.44x
RTX 5090	29.535 FW/s	34.84 FW/s	18%

The 4070 Ti SUPER comparison isolates the Ada GPU improvement before the added CPU hashing change. The 5090 shared its GPU with local pool/proof work. These are bounded search-rate comparisons, separate from full solo proving. Physical pool-search checks covered the three named cards.

Final 5090 qualification recorded 29 accepted shares with zero rejected or stale shares. The 5070 Ti run recorded 26 accepted shares and one stale rejection, plus one unclassified console rejection. Reporting accepted work alongside console speed keeps optimization tied to operational output.

5.3 From a proof milestone to a running community

The community pool launched September 6, 2026. On September 7, the public dashboard showed 66 pool blocks, 7,836 accepted shares, and one active worker at the observation time. The dashboard exposes worker activity, block history, accounting, and connection instructions. [S3]

The RC program now combines complete proof production with software distribution and real operator feedback. Members are contributing GPU results, testing releases, and helping improve installation and mining workflows. This creates a practical feedback loop between protocol engineering and the people running the network.

The operating program tracks block propagation, accepted work, restarts, wallet behavior, pool accounting, and mixed hardware. The next phase extends those measurements across a broader and more sustained participant base.

6. Ledger, network, and operator software

6.1 Canonical settlement

The ledger uses a UTXO model. Ordinary outputs are controlled by 32-byte x-only secp256k1 public keys and 64-byte BIP340 Schnorr signatures. Inference channels use a structured consensus lock. Transactions commit to the full 32-byte network identifier, and identifiers and signatures use domain-separated hashing.

Validation checks canonical encoding, available and unique inputs, signatures, maturity, exact value conservation, output bounds, and channel settlement rules. The standard transaction-frame limit is 64 KiB, with 128 inputs and 128 outputs per transaction. Blocks allow up to 1,024 transactions subject to aggregate input, output, signature, and frame limits.

6.2 Difficulty and fork choice

The target block interval is 60 seconds. Difficulty uses up to 180 header-work records, with an observed span clamped between one-third and three times the expected span. Checked wide-integer arithmetic produces the target that every validating node derives independently.

```

average_target = floor(sum(T_i) / Nw)
expected_span  = (Nw - 1) * 60
actual_span    = clamp(t_last - t_first,
                      expected_span / 3, 3 * expected_span)
next_target    = min(pow_limit,
                    max(1, floor(average_target * actual_span
                                / expected_span)))
work(T)        = floor(2^256 / (T + 1))

```

Effective median timestamps enter retarget history. Blocks must advance beyond the median of the preceding 11 timestamps and satisfy the specified future-time bound. Cumulative work uses 512 bits. A fully validated branch activates when its cumulative work is strictly greater; equal-work branches retain the active tip. Settlement assurance grows with confirmations under the Nakamoto-style work model.

6.3 Durable operations

Nodes validate block bodies before state admission, append checksummed records, synchronize durable writes, and reconstruct state through deterministic replay. The mempool applies explicit limits and deterministic ordering. Authenticated fast-start checkpoints improve recovery while preserving the validated state boundary.

RC networking uses configured peers, network fingerprints, bounded admission, and complete proof verification. The public-network roadmap expands discovery, propagation, peer policy, and long-history storage. These improvements extend the current working network toward larger operator deployments.

6.4 Wallet and release experience

RC5 provides GUI send/receive, encrypted key handling, native backup and restore dialogs, and wallet-integrated solo mining. Runtime launchers authenticate downloaded artifacts and support resumable setup. Release manifests carry detached signatures and checksums, while build metadata identifies source and binary inputs. [S1-S2]

The wallet and pool use the same consensus fee floor. Coinbase maturity is displayed in the wallet, allowing operators to distinguish newly mined outputs from spendable funds. Consolidation combines mature outputs into a self-owned output, less the burned transaction fee, to keep later spends within input limits.

7. CMFD monetary design

7.1 A calculable issuance schedule

One CMFD equals 100,000,000 atomic units. The bootstrap lasts 2,628,000 blocks, equivalent to five 365-day years at target spacing. It begins at 500 CMFD per block and declines linearly at each height.

```

N  = 2,628,000
R0 = 50,000,000,000 atoms
R(h) = floor(R0 * (N - h + 1) / N), 1 <= h <= N
R(h) = 500,000,000 atoms,          h >= N + 1

```

The permanent tail is 5 CMFD per block, paid entirely to miners. It begins after the final bootstrap block. Height-based rules make issuance independently calculable from chain state.

7.2 Visible network-building allocations

During the bootstrap, each block allocates 25% to stewardship, 5% to the community, and the remaining 70% to the miner. Rounding remainder belongs to the miner, keeping the outputs equal to the scheduled subsidy.

```

steward  = floor(R(h) * 25 / 100)
community = floor(R(h) * 5 / 100)
miner    = R(h) - steward - community

```

These allocations fund a defined network-building period: mining participation, continuing engineering, and community development. They are visible in every bootstrap block and end when the miner-only tail begins. Miner outputs mature after 100 blocks; steward and community outputs can be consumed in a later block without that miner-maturity delay. Destinations and percentages are committed network parameters.

Recipient	Aggregate bootstrap CMFD
Miners	459,900,175.01312000
Steward	164,250,062.48688000
Community	32,850,012.48688000
Total	657,000,249.98688000

The schedule provides transparent continuing funding through per-block distribution rather than an upfront token sale. Stewardship reporting and key operations are part of the network's operational development program.

7.3 Fee burning and the long-term security budget

Every ordinary transaction fee and inference-channel close fee is burned. RC5 enforces a minimum of 0.1 CMFD, or 10,000,000 atoms, per non-coinbase transaction on RCNet-1, including normal spends and channel closes. [S1, S8]

```
fee_burned = sum(input values) - sum(output values)
net_protocol_supply = gross_issuance - cumulative_burned_fees
```

Scheduled subsidy funds mining; transaction activity reduces outstanding supply through burning. At target spacing, the 5 CMFD tail issues 2,628,000 CMFD per year, approximately 0.4% of aggregate bootstrap issuance in its first year. Its percentage contribution declines as the gross supply base grows.

Fee burning connects network use to supply reduction, while the tail maintains an explicit ongoing miner budget. Net supply depends on both issuance and actual transaction activity. The design makes that relationship calculable rather than dependent on a discretionary monetary decision.

8. Direct inference settlement

8.1 A service model built around incremental delivery

The proposed inference market lets customers buy service from the provider that accepts and executes their job. CMFD payment channels combine prepaid funding, signed cumulative metering, and on-chain settlement. A provider can serve inference independently of whether it is also mining.

This is a commercially useful separation: consensus supplies neutral ordering and settlement, while providers compete on models, price, latency, availability, and customer experience. The same ledger can support varied inference runtimes because block validity depends on payment authorization rather than a particular model's response.

8.2 Exact terms and pricing

A channel commits to network and job identifiers, customer and provider keys, model/runtime/input digests, deposit, close fee, base price, input/output token prices, token limits, output chunk size, and refund height.

```
provider_payment
= base_price
+ ceil(input_tokens * input_price_per_1000 / 1000)
+ ceil(output_tokens * output_price_per_1000 / 1000)

deposit = provider_payment + customer_refund + close_fee_burn
```

All pricing is integer arithmetic in CMFD atoms. Chunk size is a negotiated channel term. The channel's exact conservation rule makes the provider payment, unused balance, and burned fee explicit at settlement.

8.3 Progressive authorization

The customer funds the channel and signs an initial cumulative state covering the agreed initial charges. The provider delivers an output chunk and a signed receipt. The customer then authorizes the next cumulative state. This repeats until completion or the configured limit.

The provider settles a correctly priced customer-signed state, normally the one carrying the largest authorized payment. Consensus verifies the signatures and allocation; the provider can close without another customer interaction. At the refund height, the customer can execute the timeout-refund path. Both terminal paths retire the channel identifier.

A provider receipt authenticates its signed token counts and rolling output digest. Receipt chaining supports the off-chain delivery flow; the customer-signed state is the on-chain spending authorization. Service-quality validation belongs to the application layer, where runtimes, repeatable jobs, provider assessment, and delivery checks can be selected for each use case.

The initial authorization covers the base price, metered inputs, the first output chunk, and the close fee. Each subsequent authorization adds at most one configured chunk of exposure. This supports finer-grained commercial interaction than a single unbounded service commitment.

8.4 Product development sequence

Implemented libraries provide channel identifiers, terms, exact pricing, signed customer states, provider receipts, settlement signatures, timeout refunds, close-fee burning, and consensus locks. The marketplace product roadmap connects these primitives to provider discovery, signed quotes, job transport, inference execution, streaming, and wallet workflows.

The first service pilot should use a defined model/runtime pair and a small provider cohort. Expansion can follow demonstrated delivery quality, repeat customer demand, and stable settlement. Provider reputation, richer commercial terms, and additional runtime choices then become extensions of a working transaction flow.

9. Engineering assurance and execution roadmap

9.1 Assurance by design

Engineering objective	Mechanism
Reproducible acceptance	Canonical objects, domain separation, fixed transcript and field encodings
Block-specific computation	Challenge binds network, model, template, target, and nonce
Independent validation	CPU candidate verification and receiving-node proof verification
Artifact consistency	Signed manifests, pinned identities, authenticated model and proving inputs
Controlled resource use	Frame limits, bounded counts, queue limits, and exact proof topology
Durable accounting	Checksummed storage, deterministic replay, reorganization-aware pool ledger
Predictable economics	Height-based issuance, exact allocation, enforced fee burning

The consensus model relies on consistent rule enforcement, the cryptographic properties of its selected primitives, and sufficient honest work and network delivery for cumulative-work convergence. Independent review and broad operating evidence strengthen this foundation as the network expands.

9.2 Milestones that extend commercial readiness

Phase	Deliverable	Evidence of completion
Current: RC operation	Public pool, usable node/wallet/miner releases, complete V4 proof	Accepted blocks and shares, signed artifacts, operator feedback
Performance and reach	Faster proving, efficient pool search, broader GPU/driver coverage	Physical-device results, accepted work, power and latency records
Network scale	Improved propagation, discovery, recovery, and storage	Sustained multi-operator runs, restart and synchronization results
Independent assurance	Protocol reproduction, cryptographic and implementation review	Published findings, reviewed resolutions, compatibility vectors
Mainnet preparation	Final parameters, operational ownership, distribution and launch plan	Complete release package, documented readiness, advance launch notice
Service-market pilot	Quotes, execution, streaming, and wallet-integrated channels	Completed paid-service flows, delivery metrics, repeat pilot usage

The release program develops reproducible-build evidence, artifact provenance, operator documentation, and wallet/pool key procedures alongside network testing. Mainnet preparation includes independent external review and a clear transition from RC participation to a fresh chain.

9.3 How to assess progress

The relevant question is whether the next release improves deployment, validated work, or service delivery. A useful scorecard tracks accepted-work efficiency, complete proof latency, time to first successful setup, active independent operators, propagation and recovery, and eventually completed service jobs and repeat usage.

This evidence connects technical progress to adoption. It also directs resources toward the engineering that improves the experience of the next miner, operator, developer, or customer.

10. Conclusion

Common Foundry combines a differentiated computational workload with independently verifiable settlement and an explicit route toward direct GPU services. Its core proposition is concrete: build the network around matrix-capable hardware, make valid work independently checkable, and give service applications programmable payment rules suited to incremental delivery.

The RC milestone establishes a working base. The complete ForgeMatrix proof is operational, consumer-card proof feasibility is measured, pool-search performance is improving, and community mining is producing observable blocks and shares. These achievements move the project from architecture into an operating product cycle.

The next opportunity is to turn that foundation into a broader operator ecosystem and a focused inference-service pilot. Common Foundry's case rests on the technology, the people running it, and a development sequence that makes each step visible and measurable.

Appendix A. Technical parameter reference

Parameter	Value
Active network / release	RCNet-1 / RC5; pool-miner update 2
ForgeMatrix algorithm / proof version	4 / 1
Batch / dimension / layers	128 / 4,096 / 384
Banks / layers per bank	3 / 128
Weight bytes	6,442,450,944
Base-table bytes	524,288
Active field	KoalaBear, 2,130,706,433
Final activation	524,288 canonical fields, 2,097,152 bytes
Transparent proof	12,025,320 bytes
Proof / block frame caps	13 MiB / 16 MiB
Transaction frame cap	64 KiB
Target interval / difficulty window	60 seconds / up to 180 records
Miner coinbase maturity	100 blocks
Bootstrap length	2,628,000 blocks
Initial / tail reward	500 / 5 CMFD
Bootstrap split	70% miner / 25% steward / 5% community
RC5 minimum burn	0.1 CMFD per non-coinbase transaction

Appendix B. Evidence and source notes

S1. RC5 release and implementation. Common Foundry v0.1.0-rc.5 release notes; wallet/mining setup, native backup/restore, and enforced minimum fee. Public release:

<https://github.com/JustAResearcher/CommonFoundry-Binaries/releases/tag/v0.1.0-rc.5>

S2. Pool miner 2. Common Foundry v0.1.0-rc.5-miner.2 release notes and packaged performance records; configuration-specific search rates, accepted shares, and output-equivalence checks.

<https://github.com/JustAResearcher/CommonFoundry-Binaries/releases/tag/v0.1.0-rc.5-miner.2>

S3. Community pool. Live pool dashboard observed September 7, 2026: 66 pool blocks, 7,836 accepted shares, one active worker. Operating counters are a dated snapshot. <https://pool.commonfoundry.ai/>

S4. Community launch. Common Foundry Discord announcements and general-channel launch messages, September 6, 2026. Community entry: <https://discord.gg/XGuutqWMWP>

S5. ProductionV4 binding specification. Project source: docs/consensus/production-v4-core-spec-v1.md and crates/cmfd-consensus/src/forgematrix_v4.rs. Covers challenge, final activation, work digest, transcript statement, and algorithm identity.

S6. ProductionV4 proof algebra. Project source: docs/consensus/production-v4-proof-algebra-v1.md and the accompanying message-order manifest. Covers KoalaBear arithmetic, extension, relation equations, claim routing, BaseFold parameters, and proof layout. BaseFold source is pinned to revision 92b8eabaea9ab7306da5826caa700adabf7445ba.

S7. Full-proof baselines. ProductionV4 qualification recorded in the devnet.16 release notes and preceding technical edition. Online replay/proof/self-verification and complete three-process results retain separate measurement scopes. <https://github.com/JustAResearcher/CommonFoundry-Binaries/releases>

S8. Monetary and consensus implementation. Project source: crates/cmfd-consensus/src/economics.rs, chain.rs, network.rs, and difficulty.rs in the RC5 integration source. Monetary quantities in this edition follow those protocol rules.

Project source references identify implementation evidence and specification files; public release links identify distributed software. This edition updates the active V4 arithmetic description and RC5 fee rule, separates measured pool search from solo proving, and presents the inference market as the product expansion built on implemented settlement primitives.